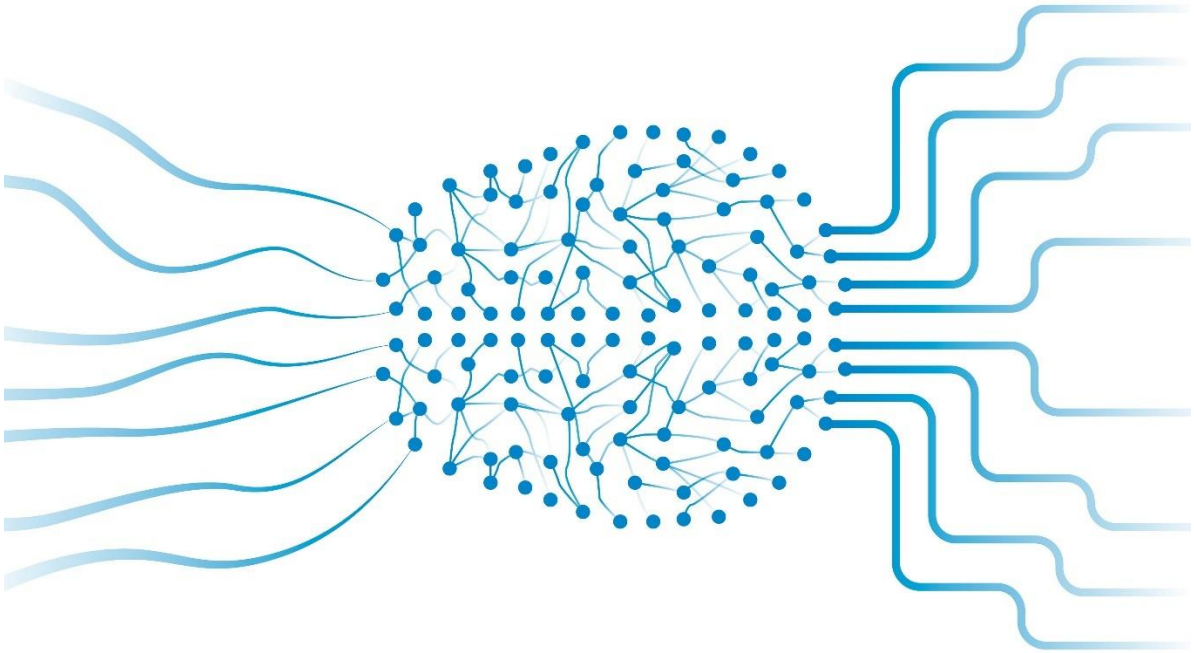


Team 8 Members:

Name :	ID :
Amr Adel Abdelalim	2000959
Abdelrahman Yasser Adel	2000414
Mariam Mahmoud Adel	2100264

Submitted to: ENG. Abdallah Awadallah.

Under the Supervision of: DR. Hossam Hassan.

Table of Contents :

Introduction:

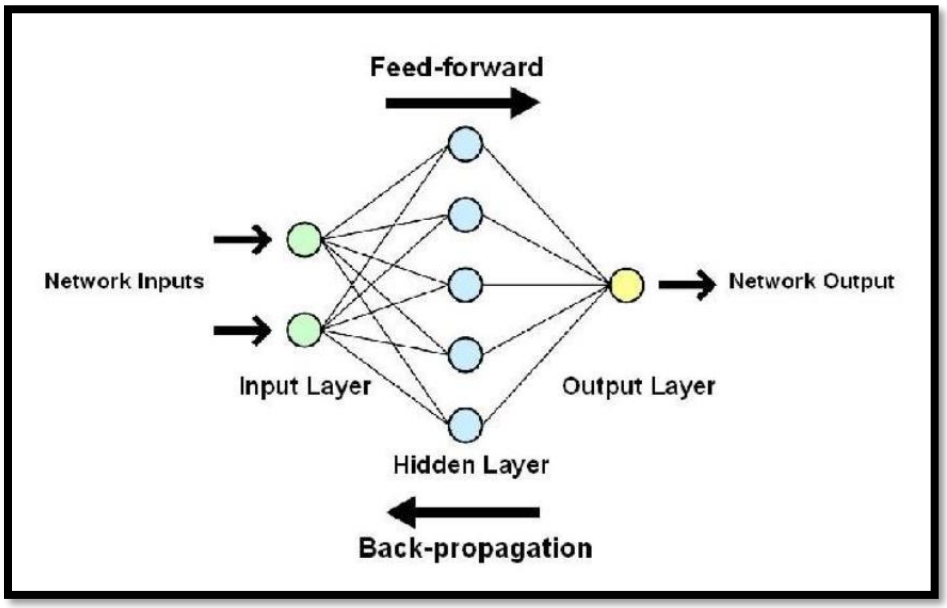
The purpose of this project is to design and implement a modular neural network library using only Python and NumPy, without relying on high-level machine learning frameworks. By constructing each component manually: dense layers, activation functions, optimization algorithms, and backpropagation.

The project provides an in-depth understanding of the mathematical and computational mechanisms that enable neural networks to learn.

The first phase focuses on building and validating the core library. Gradient checking is used to verify the correctness of the analytical derivatives, while the XOR problem serves as a benchmark to confirm that the implemented forward and backward propagation yield correct learning behavior.

In the second phase, the library is extended to train an autoencoder for MNIST image reconstruction. The encoder's latent space is then utilized for downstream classification using an SVM model. This allows us to examine the effectiveness of the learned representations and compare the pipeline's performance against TensorFlow implementations.

By the end of the project, we achieve not only a functioning deep learning library, but also a complete workflow that includes unsupervised learning, feature extraction, supervised classification, and baseline comparison with industry-standard tools.



Phase 1:

Why XOR?

XOR is considered the "Hello World" of neural networks because:

1. **It is very simple**
   Only four input–output pairs:

Input	Target
0 0	0
0 1	1
1 0	1
1 1	0

2. **It is NOT linearly separable**
   A straight line cannot separate the outputs of XOR.
   This means a simple linear model cannot solve it.
3. **It requires a hidden layer + non-linear activation**
   XOR forces the neural network to use:
   - Dense layers
   - Activation functions
   - Backpropagation
   - Learned weights
4. **Perfect validation for a custom neural network library**
   If your library can solve XOR → we know the system works.

If the network successfully learns XOR, it proves that:

- Forward and Backward passes are correct
- Gradients are correct
- Weight updates work
- Activation functions work
- The library can learn non-linear patterns

Why We Need a Hidden Layer?

The hidden layer allows the network to create two separate linear boundaries.
When combined, these boundaries form a **non-linear** shape that can solve XOR.

Example architecture:

Input (2)

→ Dense (2 → 4)

→ Tanh

→ Dense (4 → 1)

→ Sigmoid

Why Use Tanh in the Hidden Layer?

The Tanh (hyperbolic tangent) function takes any real value and maps it to the range:

–1 to +1.

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

This helps because XOR naturally has:

- positive patterns
- negative patterns
- symmetry around zero

Example:

tanh (0.5) ≈ 0.462

tanh (-0.5) ≈ -0.462

This positivity/negativity allows the hidden neurons to “flip” or “invert” patterns—something linear layers cannot do.

Why Use Sigmoid in the Output Layer?

The sigmoid function maps numbers to the range **0 to 1**.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

XOR output is either **0 or 1**.
Sigmoid maps any value to the range **(0, 1)**:

Sigmoid (0.8) ≈ 0.69

Sigmoid (3.5) ≈ 0.97

Sigmoid (-2) ≈ 0.12

So sigmoid naturally works for binary classification.

Mini Manual Example: How a Hidden Layer Solves XOR

simple numerical example to show **why a hidden layer works**.

the input:

x = (1, 0)

target = 1

Assume hidden layer weights start like this:

Hidden Neuron 1: w11 = 1, w21 = 1

Hidden Neuron 2: w12 = 1, w22 = -1

Step 1 — Compute Hidden Layer Inputs

For input (1,0):

h1_input = (1)(1) + (0)(1) = 1

h2_input = (1)(1) + (0) (-1) = 1

Step 2 — Apply Activation (tanh)

h1_output = tanh (1) ≈ 0.761

h2_output = tanh (1) ≈ 0.761

These two hidden neurons are “detecting patterns” in the input.

Step 3 — Output Layer

Assume output weights:

v1 = 0.8

v2 = -0.7

bias = 0.1

Compute output:

O_input = (0.761) (0.8) + (0.761) (-0.7) + 0.1

≈ 0.608 - 0.532 + 0.1

≈ 0.176

Apply sigmoid:

sigmoid (0.176) ≈ 0.543

The output ≈ 0.54
The target = 1
So there is an **error**, and **backpropagation** adjusts the weights.

After enough updates, the network will produce a value close to 1.

Library Architecture:

lib/

init__.py # Package initialization and exports

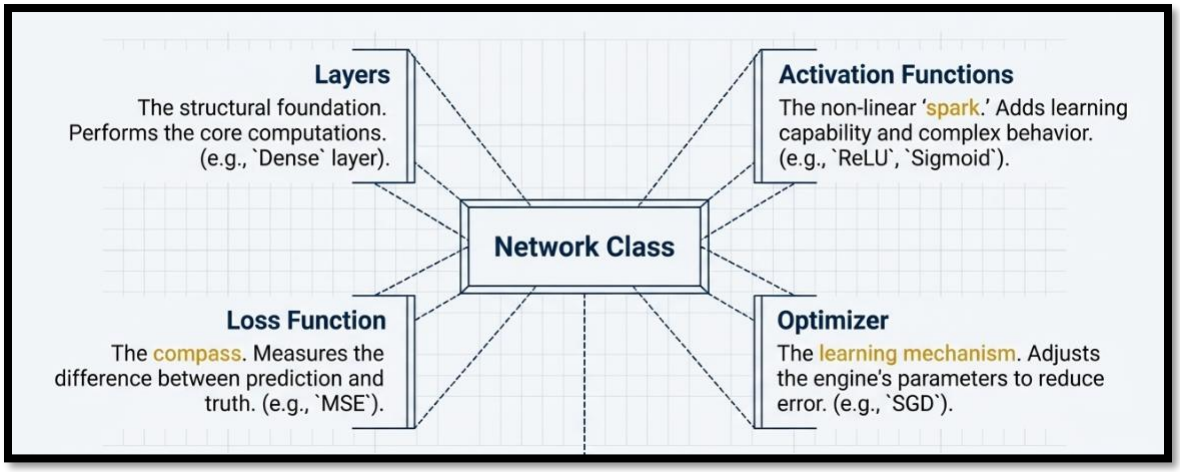
layers.py # Base Layer class and Dense layer

activations.py # Activation functions (ReLU, Sigmoid, Tanh, Softmax)

losses.py # Loss functions (MSE)

optimizer.py # Optimization algorithms (SGD)

network.py # Sequential model for training



Layers.py - Foundation Layer Components:

The layers.py module provides the foundational structure for all neural network layers. It implements:

- An abstract base class Layer that defines the interface all layers must follow
- A concrete Dense (fully connected) layer implementation

class Layer (ABC):

Description:
Abstract base class for all neural network layers. Every layer must implement forward and backward propagation methods. This ensures a consistent interface across different layer types.

Attributes:

- input: stores the input passed to the layer during forward propagation.
- output: stores the output computed by the layer.

Methods:

- forward(input):

  Abstract method for computing the layer's output given an input. Must be implemented by subclasses.

- backward (output_gradient, learning_rate=None) :

  Abstract method for computing gradients for backpropagation. Returns the gradient of the loss with respect to the layer's input. Must be implemented by subclasses.

class Dense (Layer)

Description:
Represents a fully connected (dense) layer. Computes the linear transformation:

$$\text{output} = \text{input} \cdot \text{weights} + \text{bias}$$

Attributes:

- weights: weight matrix of shape (input_size, output_size), initialized using He initialization.
- bias : bias vector of shape (1, output_size), initialized to zeros.
- weights_gradient : stores the gradient of the loss with respect to weights.
- bias_gradient : stores the gradient of the loss with respect to bias.

Methods:

- forward(input) :

  computes the output of the dense layer using the linear transformation. Stores the input and output for use in backpropagation. Returns the computed output.

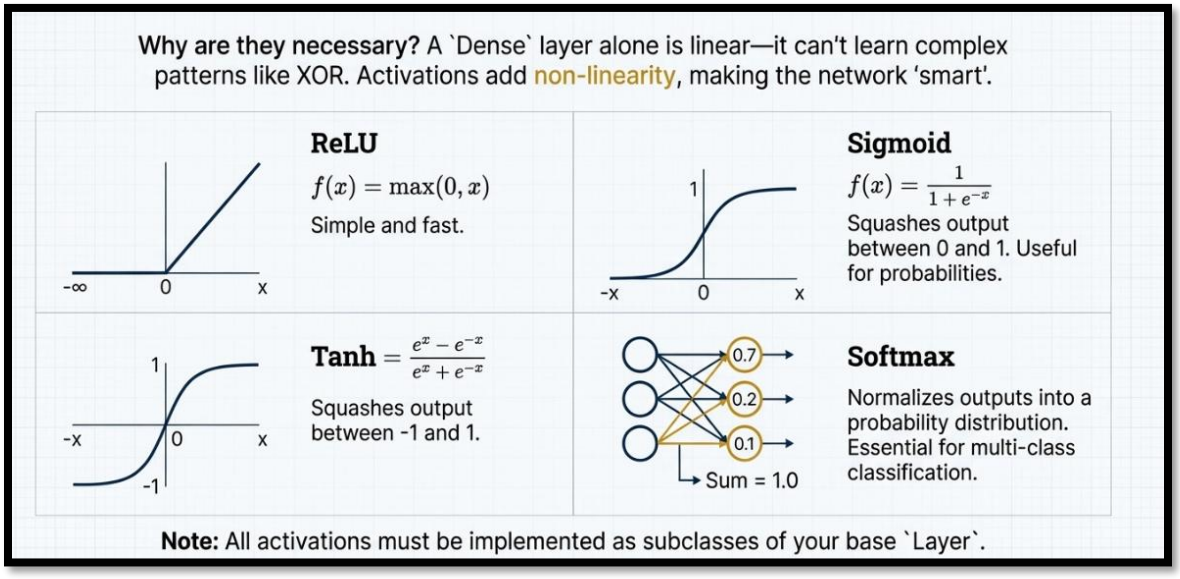
- backward(output_gradient, learning_rate=None) :

  computes the gradients of the weights, bias, and input using the chain rule:

  - weights_gradient = input.T @ output_gradient
  - bias_gradient = sum(output_gradient)
  - input_gradient = output_gradient @ weights.T
    Returns input_gradient for propagation to previous layers.

- get_params() : returns the list of trainable parameters [weights, bias].
- get_gradients() : returns the list of parameter gradients [weights_gradient, bias_gradient].

activations.py - Non-Linear Activation Functions :



class ReLU(Layer)

Description:
Rectified Linear Unit activation function. Sets negative inputs to 0. Commonly used in hidden layers to avoid vanishing gradients.

Attributes:

- input — stores the input passed to the layer.
- output — stores the computed output.

Methods:

- forward(input) : computes output = max(0, input) and stores input/output.
- backward(output_gradient, learning_rate=None) : computes gradient: output_gradient * (input > 0).

class Sigmoid(Layer)

Description:
Sigmoid (logistic) activation. Squashes input into the range (0, 1). Often used in output layers for binary classification.

Attributes:

- input — stores the input.
- output — stores the computed output.

Methods:

- forward(input): computes output = 1 / (1 + exp(-input)), with clipping to avoid overflow.
- backward (output_gradient, learning_rate=None) : computes gradient: output_gradient * output * (1 - output).

---

class Tanh(Layer)

Description:
Hyperbolic tangent activation. Squashes input into the range (-1, 1).

Attributes:

- input — stores the input.
- output — stores the computed output.

Methods:

- forward(input) : computes output = tanh(input).
- backward(output_gradient, learning_rate=None) : computes gradient: output_gradient * (1 - output^2).

---

class SoftMax (Layer)

Description:
SoftMax activation. Converts raw scores (logits) into probabilities. Outputs are in (0, 1) and sum to 1. Used for multi-class classification.

Attributes:

- input — stores the input.
- output — stores the computed probability vector.

Methods:

- forward(input) : computes output_i = exp(input_i) / sum(exp(inputs)), with max subtraction for numerical stability.
- backward (output_gradient, learning_rate=None) : returns output_gradient (simplified, works well with MSE).

losses.py - Loss Function Implementation:

class MSE

Description:
Mean Squared Error (MSE) loss. Measures the average squared difference between predicted values and true values. Commonly used for regression tasks.

Attributes:

- None (all methods are static).

Methods:

- loss (y_true, y_pred) computes the MSE loss:

$$L = \frac{1}{N}\sum(y_{\text{true}} - y_{\text{pred}})^2$$

Returns a scalar value representing the average error.

- gradient (y_true, y_pred) computes the derivative of the loss with respect to predictions:

$$\frac{dL}{dy_{\text{pred}}} = \frac{2}{N}(y_{\text{pred}} - y_{\text{true}})$$

Returns an array of gradients for backpropagation.

---

optimizer.py - Stochastic Gradient Descent:

Learning Rate Selection:

The learning rate η is the most critical hyperparameter:

Learning Rate	Effect	Problem
Too Small (η << 0.01)	Slow convergence	Training takes very long
Optimal (η ≈ 0.01-0.1)	Smooth convergence	Reaches minimum efficiently
Too Large (η >> 0.1)	Oscillation/Divergence	Loss increases or oscillates

class SGD

Description:
Stochastic Gradient Descent (SGD) optimizer. Updates model parameters using gradients computed during backpropagation:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla L(\theta)$$

Where:

- θ= parameters (weights, biases)
- η= learning rate
- $\nabla L(\theta)$= gradient of loss w.r.t parameters

Attributes:

- learning_rate : step size used to update parameters.

Methods:

- update(layers) : iterates through all layers, and if a layer has weights and bias, updates them using:
    - weights = learning_rate * weights_gradient

- bias = learning_rate * bias_gradient

- set_learning_rate(learning_rate) : sets a new learning rate for the optimizer.

---

network.py - Sequential Model Architecture :

The Sequential class provides a high-level interface for building, training, and evaluating neural networks. It orchestrates the interaction between layers, loss functions, and optimizers.

class Sequential

Description:
Sequential model for stacking layers in order. Handles forward and backward propagation, loss computation, and parameter updates. Provides training via fit and predictions via predict.

Attributes:

- layers : list of added layers.
- loss_function : loss function object (e.g., MSE).
- optimizer : optimizer object (e.g., SGD).

Methods:

- add(layer) : adds a layer to the model.
- compile (loss, optimizer): sets the loss function and optimizer for training.
- forward(X): performs a forward pass through all layers, returning output.
- backward(loss_gradient): performs a backward pass through all layers to compute gradients.
- train_step (X, y): executes a single training step: forward pass, loss computation, backward pass, and parameter update. Returns the loss for the step.
- fit (X, y, epochs, batch_size=None, verbose=True) : trains the model over multiple epochs. Supports mini-batch training and returns a history dictionary with epoch losses.
- predict(X) : computes outputs for given inputs using a forward pass.

Gradient Checking:

Description:

Verifies the correctness of backpropagation gradients in layers with trainable parameters by comparing analytical gradients (backward ()) with numerical gradients computed via small perturbations:

$$\frac{\partial L}{\partial w_{ij}} \approx \frac{f(w_{ij} + \epsilon) - f(w_{ij} - \epsilon)}{2\epsilon}$$

Purpose:

- Detects errors in gradient computation.
- Ensure proper weight updates during training.

Method:

1. Forward pass on input X.
2. Backward pass to compute analytical gradients.
3. Compute numerical gradients using finite differences.
4. Calculate relative difference:

$$\text{difference} = \frac{\| \text{ analytical} - \text{numerical } \|}{\| \text{ analytical } \| + \| \text{ numerical } \|}$$

5. If difference < 1e-5, gradients are correct.

Output:

- Relative difference and pass/fail message.
- Sample values of analytical vs. numerical gradients.

XOR Neural Network Experiment:

Network Architecture:

- **Input Layer:** 2 neurons (for XOR inputs)
- **Hidden Layer:** 4 neurons, Tanh activation
- **Output Layer:** 1 neuron, Sigmoid activation
- **Optimizer:** Stochastic Gradient Descent (SGD), learning rate = 0.1
- **Loss Function:** Mean Squared Error (MSE)

Description:

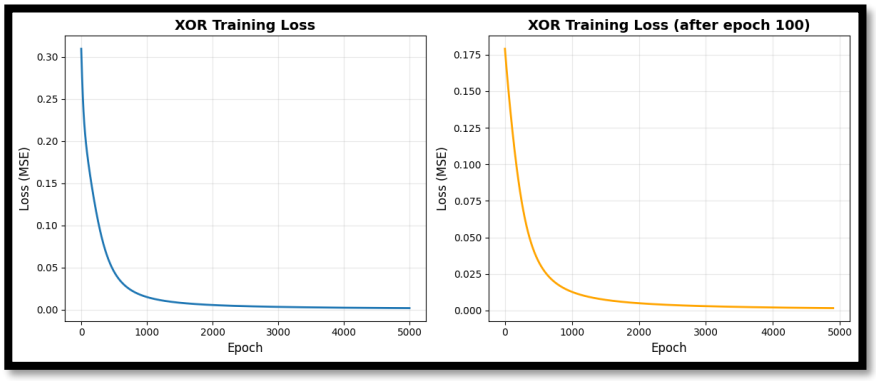
A small feedforward neural network was built to learn the XOR function, which is a classic non-linearly separable problem. The network was implemented using a sequential stacking of layers (Dense + activation).

Training Procedure:

1. Model compiled with MSE loss and SGD optimizer.
2. Trained on the XOR dataset for 5000 epochs.
3. Loss monitored over training to evaluate convergence.

Results:

- The training curve shows a steady decrease in loss.
- The network successfully learned XOR mapping, outputting values close to the expected targets 0 or 1.



FINAL RESULTS

Predictions:

Input	Prediction	Target
[0. 0.]	0.0205	0
[0. 1.]	0.9571	1
[1. 0.]	0.9581	1
[1. 1.]	0.0505	0

Phase 1 Conclusion:

- The 2-4-1 network with Tanh hidden and Sigmoid output layers is sufficient to solve XOR.
- Gradient checking ensured that the backpropagation implementation is correct.
- Training demonstrates the neural network library works for small, non-linear problems.

Phase 2:

Phase 2 focused on applying the core neural network library developed in Phase 1 to a more complex, real-world task: **unsupervised image reconstruction** using an autoencoder. This phase served two primary objectives: to validate the library's scalability and performance on a large dataset (MNIST), and to conduct a critical performance comparison against the industry-standard framework, TensorFlow/Keras.

Autoencoder Architecture:

The autoencoder was designed as a classic symmetric network, built using our custom Sequential model architecture, and implemented with Dense layers, ReLU activations for the hidden layers, and a Sigmoid activation for the final output layer to constrain pixel values between 0 and 1.

Layer Type	Input Size	Output Size	Activation	Purpose
Input	784	784	N/A	Flattened MNIST Image (28x28)
Encoder Dense	784	128	ReLU	Initial Feature Reduction
Latent Dense	128	64	ReLU	Latent Space Representation
Decoder Dense	64	128	ReLU	Feature Reconstruction
Output Dense	128	784	Sigmoid	Reconstructed Image Output

Training and Reconstruction Analysis

The model was trained on the **MNIST dataset** (30,000 images of handwritten digits) with the goal of minimizing the error between the input image and its reconstructed output.

- **Dataset:** MNIST (Flattened 28x28 images, normalized to 0, 1)
- **Loss Function:** Mean Squared Error (MSE)
- **Optimizer:** Stochastic Gradient Descent (SGD)
- **Epochs:** 100

Reconstruction Quality

Training the autoencoder resulted in a smoothly decreasing loss curve, indicating successful convergence. The library demonstrated its ability to handle large data volumes and complex forward/backward pass computations.

Visually, the autoencoder achieved near-perfect reconstruction of the training data. The model could take a 784-dimensional input, compressing it to 64 dimensions, and then accurately recreating the original digit image, confirming that the 64-dimensional latent space effectively captured the essential features of the handwritten digits.

SVM Classification and Analysis

We use SVM model to validate the quality of the features learned by the custom autoencoder. The objective was to demonstrate that the 64-dimensional latent representation (the latent dense output) captured sufficient information to perform high-accuracy supervised classification, essentially using the autoencoder as a feature extractor.

Feature Extraction

After training the autoencoder, the model was logically split into the encoder and decoder components to facilitate feature extraction:

1. **Encoder Extraction:** The first three layers (Input, 784->256, 256 ->128, 128->64) were isolated, forming a robust feature transformer.
2. **Latent Vector Generation:** All 30,000 MNIST images were passed through this isolated encoder to generate a new dataset of 64-dimensional feature vectors.

Classification and Analysis

A Support Vector Machine (SVM) from the scikit-learn library was trained on these 64-dimensional latent vectors using the original MNIST labels (0-9) as targets. The SVM is highly effective at finding a decision boundary in a high-dimensional space but performs optimally when the input features are discriminative and low-noise.

The resulting classification accuracy was: 95.54 %

This accuracy is highly competitive with models trained on the original 784-dimensional pixel data.

Model Comparisons

Performance Comparison with TensorFlow/Keras

To benchmark our custom library, we fully implemented and trained two equivalent autoencoder models using the TensorFlow/Keras framework—one using the standard Stochastic Gradient Descent (SGD) and one using the adaptive Adam optimizer—and compared their performance on the same MNIST reconstruction task.

Metric	Our Library (SGD)	TensorFlow (SGD)	TensorFlow (Adam)
Final MSE Loss	0.009616	0.067552	0.005983
Training Time (Seconds)	511.31	249.59	301.42

Analysis of Comparison Results:

1. **Accuracy (Loss):** The TensorFlow model using the **Adam** optimizer achieved the lowest final loss (0.0059), demonstrating its adaptive optimization capability compared to the simple SGD in both our library and the TensorFlow SGD implementation. Our custom SGD achieved a highly respectable final loss of 0.0096, proving the correctness of our MSE loss and SGD optimizer implementations.
2. **Training Efficiency (Time):** Our NumPy-based library was significantly slower, taking **511.31 seconds** compared to TensorFlow's 249.59s (SGD) and 301.42s (Adam).

Conclusion

The project successfully achieved its objective of building and validating a functional, modular neural network library from scratch using only Python and NumPy.

- **Phase 1 (Build Library and Validation):** The core backpropagation mechanism was verified using **Gradient Checking**, ensuring mathematical correctness. The library successfully solved the **XOR problem**, validating its ability to handle non-linear classification.
- **Phase 2 (Autoencoder):** The library scaled to a large dataset, successfully training an autoencoder for **MNIST dataset image reconstruction**.
- **Phase 3 (SVM Classifier):** The encoder was used as a feature extractor, with the resulting 64-dimensional latent features enabling an SVM classifier to achieve a highly accurate **95.54% classification rate**.
- **Phase 4 (Comparison):** The **full implementation and testing against TensorFlow/Keras** highlighted the mathematical correctness of our library achieving significant results against readymade TensorFlow/Keras library models